FINAL STUDY MATERIAL · IBM CONFLUENT · JD 129283

🎯 Confluent Interview Complete Prep Guide

Novice se Advanced tak — Concurrency, DSA, LLD, HLD, Kafka aur Behavioral, sab ek jagah

Ye document kya hai: Is poori chat me jo bhi research, JD analysis, aur question-bank bana — sab ko ek 4-din ke study plan me organize kiya gaya hai. Har topic novice explanation se shuru hota hai, phir code, phir interview-level depth.

Role reminder: Senior Software Engineer (L3), Confluent KORA — **Marketplace & Hub domain**. Remote, Bangalore-based. 4-7 YOE band. Java/Scala/Go expected — tu Java me hi rahega.

ROADMAP

4-Din Ka Study Plan

Har din ~3-4 ghante, priority order me

Priority logic: Dono candidate experiences me **Concurrency ka dedicated round hamesha tha** — isliye wo #1 priority hai. LRU Cache dono me aaya — #2. HLD open-ended hota hai — framework yaad rakhna kaafi hai.

⌨ Din 1 — Concurrency Foundations

- Part 1 poora padho (threads, locks, atomics, executors)
- Thread-safe Ring Buffer khud se likho, bina dekhe
- Har chapter ka "Khud Test Kar" attempt karo

⌨ Din 2 — Concurrency Problems + LRU

- Sudoku Validator (multithreaded) — khud likho
- Job Scheduler — design + code
- LRU Cache classic → phir TTL variation → phir heap variation

⌨ Din 3 — DSA + LLD Round

- Session Manager, Time Window Map, tail -f, Large File
- Number of Atoms (stack+map parsing)
- Map+Tree DSA patterns — complexity bolne ki practice

⌨ Din 4 — HLD + Domain + Behavioral

- Key-Value Store HLD — requirement gathering framework ratna
- Kafka internals + KORA/Marketplace domain revision
- 3-4 STAR stories bolke practice karo (loudly, timer ke saath)

PART 0

Interview Structure — Kya Expect Karna Hai

Do candidate experiences cross-referenced

#	Round	Focus
1	Phone Screen	DSA + Concurrency mix (ya LRU variation)
2	DSA Round	Map/Tree based, complexity + test cases pe zor
3	LLD / Concurrency	Multithreaded design (Sudoku validator, Job Scheduler)
4	System Design	Open-ended (KV Store) — requirement gathering evaluate hota hai
5	Behavioral	Values, cultural fit, open discussion

Teen confirmed patterns:
1. **Concurrency ka dedicated round har baar** — ye sabse zyada weight wala topic hai
2. **LRU Cache dono experiences me** — different variations, core question hai
3. **HLD round me fixed problem nahi** — tujhe requirements pooch-pooch ke khud scope banana hai

Abhi tera current stage: HackerRank Coding Assessment (Round 0, screening). Ye clear hone ke baad hi upar wale 5 rounds shuru honge. Isliye Din 1-3 ka DSA/coding focus abhi zyada important hai; HLD/Behavioral thoda baad me bhi chalega.

PART 1 · HIGHEST PRIORITY

Concurrency — Novice se Advanced

Har interview me dedicated round hai. Isko sabse achhe se pakadna hai.

1 Race Condition — Sabse Basic Problem

Socho ek bank account me ₹100 hain. Do log ek saath ATM se ₹100 nikaal rahe hain. Dono ka code padhta hai "balance = 100", dono sochte hain "haan paisa hai", dono nikaal lete hain — bank ka ₹100 ka nuksaan. Ye ***race condition*** *hai — do threads ek hi shared cheez ko ek saath chhed rahe hain.*

Shared Mutable State = koi bhi variable/object jo (a) ek se zyada threads dekhte hain, aur (b) badalta hai. Concurrency ki har problem isi ek cheez se shuru hoti hai.

Interview me sabse pehla step hamesha: "Pehle main identify karunga ki shared mutable state kya hai" — ye line bolna hi tujhe senior candidate dikhata hai.

```
// UNSAFE
class Counter {
    private int count = 0;
    public void increment() {
        count++;   // ye 3 operations hai: read,
add, write
    }
}
```

Kyun tuta: `count++` dikhne me ek line hai, but CPU ke liye 3 steps hain — **read → add → write**. Do threads beech me ghus sakte hain. Isse **"non-atomic operation"** kehte hain.

🧠 Khud Test Kar

Q: count++ atomic kyun nahi hai?

> A: Kyunki ye internally teen alag operations hai (read, increment, write) — beech me doosra thread interrupt kar sakta hai aur stale value use kar sakta hai.

2 Locks — synchronized aur ReentrantLock

synchronized — Java ka built-in lock. Ek time pe sirf ek thread andar ja sakta hai. Simple, automatic release (exception aane pe bhi).

```
public synchronized void increment() {
    count++;
}

// ya block-level (better — scope chhota)
public void increment() {
    synchronized (this) {
        count++;
    }
}
```

ReentrantLock — manual lock, zyada power deta hai:
• `tryLock(timeout)` — lock na mile toh wait mat karo (deadlock se bachne ka tarika)
• `lockInterruptibly()` — waiting ke beech thread interrupt ho sakta hai
• Fairness policy set kar sakte ho (jo pehle aaya use pehle lock)

```
private final ReentrantLock lock = new
ReentrantLock();

public void increment() {
    lock.lock();
    try {
        count++;
    } finally {
        lock.unlock();   // finally me hamesha!
    }
}
```

Trap: ReentrantLock me `unlock()` hamesha `finally` block me hona chahiye. Bhool gaya toh exception aane pe lock kabhi release nahi hoga — poora system hang. `synchronized` me ye problem nahi hoti (JVM khud release karta hai).

ReadWriteLock — advanced optimization. Agar data **zyada padha jaata hai, kam likha jaata hai** (jaise ek config cache), toh multiple readers ek saath allow karo, sirf writer ko exclusive lock do. Throughput bahut badh jaata hai.

Ye line interview me bolna: "Main lock ka scope minimum rakhunga — poora method synchronize karne ke bajaye sirf critical section, taaki contention kam ho."

Khud Test Kar

Q: ReadWriteLock kab useful hai?

> A: Jab reads writes se bahut zyada hon — multiple readers parallel chal sakte hain, sirf write ke time exclusive lock lagta hai.

3 Atomics aur CAS — Lock ke Bina Safety

*Lock lena mehnga hai — thread ko rukna padta hai, OS ko context-switch karna padta hai. Kya koi tarika hai bina rukey safe rehne ka? Haan — **CAS (Compare-And-Swap)**.*

CAS ka idea: "Main sochta hoon value abhi 5 hai. Agar sach me 5 hai, toh use 6 kar do. Agar kisi aur ne beech me badal diya, toh mujhe bata do — main dobara try karunga."

Ye ek single CPU instruction hoti hai — isliye atomic hai, koi lock nahi chahiye.

```
AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet();   // thread-safe, no lock

// CAS manually
int old, next;
do {
    old = count.get();
    next = old + 1;
} while (!count.compareAndSet(old, next));
```

Cheez	Lock	CAS/Atomic
Blocking?	Haan, thread rukta hai	Nahi, retry karta hai
Best when	Contention zyada	Contention kam
Complexity	Simple	Retry logic sochna padta hai

Advanced trap — ABA problem: CAS sirf value dekhta hai. Agar value A → B → wapas A ho jaaye, CAS ko lagega "kuch nahi badla" — but actually badla tha. Solution: `AtomicStampedReference` (value ke saath ek version-stamp bhi rakho).

volatile keyword: Ye **atomicity nahi** deta, sirf **visibility** deta hai — matlab ek thread ne value badli toh doosre thread ko turant dikhegi (CPU cache se stale value nahi milegi). `volatile boolean running` flag ke liye perfect hai, but `volatile int count++` abhi bhi unsafe hai.

🧠 Khud Test Kar

Q: volatile aur AtomicInteger me kya farak hai?

> A: volatile sirf visibility guarantee karta hai (latest value dikhegi), atomicity nahi. AtomicInteger visibility + atomicity dono deta hai (CAS ke through).

4 ExecutorService — Threads Manage Karna

*Har kaam ke liye `new Thread()` banana bura hai — thread banana mehnga hai, aur 10,000 threads bana diye toh system mar jaayega. Solution: ek **thread pool** — kuch fixed threads jo baar-baar reuse hote hain.*

```
ExecutorService pool =
Executors.newFixedThreadPool(4);

// Runnable — koi return value nahi
pool.submit(() -> System.out.println("kaam"));

// Callable — return value deta hai
Future<Integer> f = pool.submit(() -> 42);
int result = f.get();   // blocking wait

pool.shutdown();   // naye tasks band, purane
complete honge
pool.awaitTermination(10, TimeUnit.SECONDS);
```

Teen zaroori types:
• `newFixedThreadPool(n)` — fixed size, predictable
• `newCachedThreadPool()` — zaroorat ke hisaab se badhta hai (risky agar load unbounded ho)
• `newScheduledThreadPool(n)` — delay ya repeat ke saath tasks (Job Scheduler ke liye yehi)

Future vs CompletableFuture:
`Future.get()` blocking hai — thread ruk jaata hai. `CompletableFuture` non-blocking chaining deta hai (`thenApply` , `thenCombine`), production code me isse better hai.

Interview trap: Agar tu `pool.shutdown()` nahi bulayega, JVM exit nahi hoga (non-daemon threads chalte rehte hain). Ye choti si baat interviewer notice karta hai.

🧠 Khud Test Kar

Q: Runnable aur Callable me kya farak hai?

> A: Runnable kuch return nahi karta aur checked exception throw nahi kar sakta. Callable value return karta hai (Future ke through) aur exception throw kar sakta hai.

5 Concurrent Collections

Chahiye	Use karo	Kyun
Thread-safe map	ConcurrentHashMap	Segment/bucket-level locking, poora map lock nahi hota
Producer-consumer	BlockingQueue (LinkedBlockingQueue)	put/take khud block karte hain, manual wait/notify nahi chahiye
Read-heavy list	CopyOnWriteArrayList	Har write pe copy banti hai — reads bilkul lock-free
Priority + concurrency	PriorityBlockingQueue	Job Scheduler ke liye ideal

Trap: `Collections.synchronizedMap()` aur `ConcurrentHashMap` ek jaise nahi hain. Pehla poore map pe ek hi lock lagata hai (slow), doosra fine-grained locking karta hai (fast). Interview me hamesha ConcurrentHashMap prefer karo aur ye reason batao.

ConcurrentHashMap ka atomic helper: `computeIfAbsent`, `merge`, `putIfAbsent` — ye check-then-act ko atomic banate hain. Agar tu `if (!map.containsKey(k)) map.put(k, v)` likhega, wo race condition hai — `putIfAbsent` use kar.

🧠 Khud Test Kar

Q: BlockingQueue producer-consumer me kaise help karta hai?

> A: Queue khaali ho toh take() khud wait karta hai, full ho toh put() khud wait karta hai — manual wait/notify aur uske bugs se bach jaate hain.

6 Deadlock, Livelock, Starvation

Deadlock: *Thread A ne Lock1 liya aur Lock2 ka wait kar raha hai. Thread B ne Lock2 liya aur Lock1 ka wait kar raha hai. Dono hamesha ke liye atke — koi aage nahi badhega.*

Deadlock ki 4 shartein (sab ek saath honi chahiye):
1. Mutual exclusion (resource shared nahi ho sakta)
2. Hold and wait (ek lock pakde hue doosre ka wait)
3. No preemption (zabardasti lock cheena nahi ja sakta)
4. Circular wait (A→B→A ka cycle)

Koi ek bhi todo, deadlock nahi hoga.

Practical solutions (interview me ye bolo):
• **Lock ordering** — sabhi threads locks ek hi fixed order me lein (circular wait toot gaya)
• **tryLock with timeout** — na mile toh chhod do aur retry karo (hold-and-wait toot gaya)
• **Lock scope chhota rakho** — jitni der lock pakda, utna risk

Problem	Kya hota hai
Deadlock	Sab atke, kuch nahi ho raha
Livelock	Sab kaam kar rahe hain but progress zero (dono baar-baar peeche hat rahe hain)
Starvation	Ek thread ko kabhi chance hi nahi milta (baaki hamesha jeet jaate hain)

🧠 Khud Test Kar

Q: Lock ordering deadlock kaise rokta hai?

> A: Circular wait condition todta hai — agar sab threads locks ek hi order me lete hain, A→B→A jaisa cycle ban hi nahi sakta.

PART 2

Concurrency Coding Problems

LLD/Concurrency round me yahi type aata hai

7 Sudoku Validator — Multithreaded

9x9 Sudoku board valid hai ya nahi check karna hai. Rules: har row me 1-9 unique, har column me 1-9 unique, har 3x3 box me 1-9 unique. Total 27 independent checks (9 rows + 9 cols + 9 boxes) — ***ye sab ek doosre se independent hain, isliye parallel chal sakte hain!***

Kyun ye problem multithreading ke liye perfect hai: 27 tasks, koi shared mutable state nahi (sab sirf board **padh** rahe hain, likh koi nahi raha). Read-only shared data automatically thread-safe hota hai — ye baat interview me zaroor bolna.

```
class SudokuValidator {
    private final int[][] board;

    public SudokuValidator(int[][] board) {
        this.board = board;
    }

    public boolean isValid() throws Exception {
        ExecutorService pool =
Executors.newFixedThreadPool(9);
        List<Callable<Boolean>> tasks = new
ArrayList<>();

        for (int i = 0; i < 9; i++) {
            final int idx = i;
            tasks.add(() -> checkRow(idx));
            tasks.add(() -> checkCol(idx));
            tasks.add(() -> checkBox(idx));
        }

        try {
            // invokeAll saare tasks chalata hai,
sab complete hone tak wait
            List<Future<Boolean>> results =
pool.invokeAll(tasks);
            for (Future<Boolean> f : results) {
                if (!f.get()) return false;
            }
            return true;
```

```
        } finally {
            pool.shutdown();
        }
    }

    private boolean checkRow(int r) {
        boolean[] seen = new boolean[10];
        for (int c = 0; c < 9; c++) {
            int val = board[r][c];
            if (val == 0) continue;
            if (seen[val]) return false;
            seen[val] = true;
        }
        return true;
    }

    private boolean checkCol(int c) {
        boolean[] seen = new boolean[10];
        for (int r = 0; r < 9; r++) {
            int val = board[r][c];
            if (val == 0) continue;
            if (seen[val]) return false;
            seen[val] = true;
        }
        return true;
    }

    private boolean checkBox(int b) {
        boolean[] seen = new boolean[10];
        int startRow = (b / 3) * 3;
        int startCol = (b % 3) * 3;
        for (int r = startRow; r < startRow + 3;
r++) {
            for (int c = startCol; c < startCol +
3; c++) {
                int val = board[r][c];
                if (val == 0) continue;
                if (seen[val]) return false;
                seen[val] = true;
            }
        }
        return true;
    }
}
```

Box index ka math samajh: Box 0-8 hote hain (left-to-right, top-to-bottom). Box `b` ka starting cell = row `(b/3)*3`, col `(b%3)*3`. Jaise box 4 (beech wala) → row 3, col 3.

Follow-up jo aayega — "early exit kaise karoge?": Upar wala code saare 27 tasks complete hone ka wait karta hai, chahe pehla hi invalid nikle. Better: ek `AtomicBoolean isInvalid` shared rakho, har task shuru me

check kare "agar already invalid hai toh turant return", aur invalid milne pe set kar de. Ya `CompletionService` use karo jo pehla result aate hi de deta hai.

```
// Early-exit version ka core idea
AtomicBoolean invalid = new AtomicBoolean(false);

Callable<Boolean> task = () -> {
    if (invalid.get()) return false;   // koi aur
fail kar chuka
    boolean ok = checkRow(idx);
    if (!ok) invalid.set(true);
    return ok;
};
```

Interview me ye bolna: "27 checks independent hain aur board read-only hai, isliye koi synchronization nahi chahiye. Sirf early-exit flag ke liye AtomicBoolean use karunga."

Khud Test Kar

Q: Is problem me koi lock kyun nahi chahiye?

> A: Kyunki koi thread board ko modify nahi kar raha — sab sirf read kar rahe hain. Immutable/read-only shared data inherently thread-safe hota hai.

8 Job Scheduler with Concurrency

Ek scheduler banao jahan log tasks submit kar sakein — kuch turant chalein, kuch 5 minute baad, kuch har ghante repeat hon. Multiple threads ek saath submit kar rahe hain, aur multiple workers execute kar rahe hain.

Design ke 4 core components (ye structure bolna):

1. **Job** — task ka data (id, runnable, scheduled time, priority, recurring interval)
2. **Priority Queue** — jobs time ke order me, jo pehle chalna hai wo top pe
3. **Worker pool** — jobs ko actually execute

karne wale threads
4. **Dispatcher thread** — queue se due jobs uthake pool ko deta hai

```
class Job implements Comparable<Job> {
    String jobId;
    Runnable task;
    long scheduledTime;      // epoch millis
    long intervalMillis;     // 0 = one-time
    int priority;

    public int compareTo(Job other) {
        int c = Long.compare(this.scheduledTime,
other.scheduledTime);
        if (c != 0) return c;
        return Integer.compare(other.priority,
this.priority);
    }
}

class JobScheduler {
    private final PriorityBlockingQueue<Job> queue
        = new PriorityBlockingQueue<>();
    private final ExecutorService workers;
    private final Thread dispatcher;
    private volatile boolean running = true;

    public JobScheduler(int workerCount) {
        this.workers =
Executors.newFixedThreadPool(workerCount);
        this.dispatcher = new
Thread(this::dispatchLoop);
        this.dispatcher.start();
    }

    public void submit(Job job) {
        queue.offer(job);   // thread-safe, koi
lock nahi chahiye
    }

    private void dispatchLoop() {
        while (running) {
            try {
                Job job = queue.take();   // blocks
agar khaali
                long delay = job.scheduledTime
                             -
System.currentTimeMillis();
                if (delay > 0) {
                    queue.offer(job);     // abhi
time nahi hua, wapas daal
                    Thread.sleep(Math.min(delay,
100));
                    continue;
                }
                workers.submit(() -> runJob(job));
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
```

```
                break;
            }
        }
    }

    private void runJob(Job job) {
        try {
            job.task.run();
        } catch (Exception e) {
            // log karo — ek job fail hone se
scheduler nahi marna chahiye
        } finally {
            if (job.intervalMillis > 0 && running)
{
                job.scheduledTime =
System.currentTimeMillis()
                                        +
job.intervalMillis;
                queue.offer(job);   // recurring:
reschedule
            }
        }
    }

    public void shutdown() {
        running = false;
        dispatcher.interrupt();
        workers.shutdown();
    }
}
```

Sabse bada trap — busy-wait: Upar wale dispatchLoop me agar job ka time nahi hua toh hum wapas queue me daal ke sleep karte hain. Ye simple hai but thoda inefficient. **Behtar answer:** `DelayQueue` use karo — ye khud hi tab tak block karta hai jab tak koi element "due" na ho jaaye, koi polling nahi. Ya seedha `ScheduledThreadPoolExecutor` use karo jo ye sab built-in deta hai.

Interviewer ke likely follow-ups aur short answers:

• **"Ek job crash ho jaaye toh?"** → try-catch har job ke around, scheduler chalta rahe, failed job log/retry queue me

• **"Same job do baar na chale, guarantee?"** → ek `ConcurrentHashMap<String, Boolean> runningJobs` rakho, `putIfAbsent` se claim karo

• **"Multi-machine pe kaise scale karoge?"** → distributed lock (Redis/Zookeeper) ya DB-level row locking se job claim karo, taaki do machines ek hi job na uthayein

🧠 Khud Test Kar

Q: PriorityBlockingQueue kyun, simple PriorityQueue kyun nahi?

> A: PriorityQueue thread-safe nahi hai. PriorityBlockingQueue thread-safe hai aur take() khaali hone pe block karta hai — dispatcher ko busy-loop nahi karna padta.

9 Thread-Safe Ring Buffer (O(1) Queue + Random)

*Ek queue chahiye jisme **enqueue O(1)**, **dequeue O(1)**, aur **random() O(1)** — teeno. LinkedList me random slow (O(n) traverse), plain array me dequeue slow (O(n) shift). Solution: **circular array (ring buffer)**.*

Kaise kaam karta hai: Array ko "gol" treat karo — `head` (front) aur `tail` (end) indices ghoomti rehti hain modulo ke through.
• Enqueue → tail pe daalo, `tail = (tail+1) % cap`
• Dequeue → head se lo, `head = (head+1) % cap`
• Random → `(head + rand(0, size-1)) % cap` — direct index access
• Full ho jaaye → capacity double karo (isliye **amortized** O(1))

```
class ConcurrentRandomQueue<T> {
    private Object[] arr;
    private int head = 0, tail = 0, size = 0,
capacity;
    private final ReentrantLock lock = new
ReentrantLock();
    private final Random rand = new Random();

    public ConcurrentRandomQueue(int cap) {
        this.capacity = cap;
        this.arr = new Object[cap];
    }
```

```
    public void enqueue(T val) {
        lock.lock();
        try {
            if (size == capacity) resize(capacity *
2);
            arr[tail] = val;
            tail = (tail + 1) % capacity;
            size++;
        } finally {
            lock.unlock();
        }
    }

    @SuppressWarnings("unchecked")
    public T dequeue() {
        lock.lock();
        try {
            if (size == 0) throw new
NoSuchElementException();
            T val = (T) arr[head];
            arr[head] = null;
            head = (head + 1) % capacity;
            size--;
            return val;
        } finally {
            lock.unlock();
        }
    }

    @SuppressWarnings("unchecked")
    public T random() {
        lock.lock();
        try {
            if (size == 0) throw new
NoSuchElementException();
            int idx = (head + rand.nextInt(size)) %
capacity;
            return (T) arr[idx];
        } finally {
            lock.unlock();
        }
    }

    private void resize(int newCap) {
        Object[] newArr = new Object[newCap];
        for (int i = 0; i < size; i++)
            newArr[i] = arr[(head + i) % capacity];
        arr = newArr; head = 0; tail = size;
capacity = newCap;
    }
}
```

Step	Action	head	tail	size	Array (cap 4)
1	enq(A)	0	1	1	[A,_,_,_]
2	enq(B)	0	2	2	[A,B,_,_]
3	deq()→A	1	2	1	[_,B,_,_]

4	enq(C)	1	3	2	[_,B,C,_]

Interview trap: `resize()` sabse risky operation hai — poora array copy hota hai. Bolna zaroor: "resize ke dauraan koi doosra thread access nahi kar sakta, isliye ye lock ke andar hi hona chahiye." Aur ye bhi ki isliye complexity **amortized** O(1) hai, strict O(1) nahi.

🧠 Khud Test Kar

Q: random() O(1) kaise hai jabki head kahin bhi ho sakta hai?

> A: Underlying array hai — bas ek random offset (0 se size-1) head me add karke modulo lena hai. Direct index access, koi traversal nahi.

PART 3

DSA + LLD Problems

LRU (recurring!), parsing, file I/O, aur design problems

10 LRU Cache — Classic (Base Version)

☆ **Ye dono candidate experiences me aaya tha** — different variations me. Base version pehle perfect karo, phir variations.

Cache me limited jagah hai (capacity N). Jab full ho jaaye aur naya item aaye, **sabse purana-use-kiya-gaya** *(Least Recently Used) item nikaal do. Dono operations — get aur put — O(1) me hone chahiye.*

Do cheezein chahiye, isliye do data structures:

• **HashMap** → key se node turant milna chahiye, O(1) lookup

• **Doubly Linked List** → "recency order" maintain karna, aur kisi bhi node ko O(1) me nikaal/aage laa sakna

Head ke paas = recently used. Tail ke paas = LRU (yahi evict hoga).

DLL kyun, singly nahi? *Kyunki node ko beech se hatana hai — uske* **previous** *node ka pointer chahiye. Singly list me previous dhundhne ke liye traverse karna padega, O(n) ho jaayega.*

```
class LRUCache {
    class Node {
        int key, value;
        Node prev, next;
        Node(int k, int v) { key = k; value = v; }
    }

    private final Map<Integer, Node> map = new
HashMap<>();
    private final Node head, tail;   // dummy
sentinels
    private final int capacity;
```

```
    public LRUCache(int capacity) {
        this.capacity = capacity;
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        Node node = map.get(key);
        if (node == null) return -1;
        moveToFront(node);
        return node.value;
    }

    public void put(int key, int value) {
        Node node = map.get(key);
        if (node != null) {
            node.value = value;
            moveToFront(node);
            return;
        }
        if (map.size() == capacity) {
            Node lru = tail.prev;
            remove(lru);
            map.remove(lru.key);
        }
        Node fresh = new Node(key, value);
        map.put(key, fresh);
        addFirst(fresh);
    }

    private void remove(Node n) {
        n.prev.next = n.next;
        n.next.prev = n.prev;
    }

    private void addFirst(Node n) {
        n.next = head.next;
        n.prev = head;
        head.next.prev = n;
        head.next = n;
    }

    private void moveToFront(Node n) {
        remove(n);
        addFirst(n);
    }
}
```

Dummy head/tail ka fayda: Inke bina har remove/add me "agar ye pehla node hai toh...", "agar aakhri hai toh..." wale null-checks likhne padte. Sentinels se code clean aur bug-free ho jaata hai. **Ye baat interview me bolna — senior signal hai.**

Sabse common bug: `map.remove()` karte waqt galat key use karna. Node ke andar **key bhi store karni padti hai** (sirf value nahi) — warna evict karte waqt pata hi nahi chalega ki map se kaunsi key hataani hai. Isliye Node me `key` field zaroori hai.

Complexity: get O(1), put O(1). Space O(capacity).

Khud Test Kar

Q: Node me key store karna kyun zaroori hai, value toh hai hi?

> A: Eviction ke time hum tail se node uthate hain — us node se map ki key nikaalni padti hai taaki map.remove(key) kar sakein. Key ke bina reverse-lookup O(n) ho jaata.

11 LRU Variation A — TTL ke Saath

*Ab twist: har entry ki ek **expiry time (TTL)** bhi hai. Matlab entry do wajah se ja sakti hai — (1) capacity full hone pe LRU evict, ya (2) time khatam hone pe expire. Dono logic ek saath handle karne hain.*

Key insight: Do alag "orders" maintain karne hain —
- **Recency order** → DLL (LRU ke liye)
- **Expiry order** → alag se track karo

Sabse simple approach: **lazy expiry** — node me `expiryTime` store karo, get() ke time check karo, expired ho toh nikaal do aur -1 return karo.

```
class Node {
    int key, value;
    long expiryTime;      // naya field
    Node prev, next;
}

public int get(int key) {
    Node node = map.get(key);
```

```
    if (node == null) return -1;

    if (System.currentTimeMillis() >
node.expiryTime) {
        remove(node);           // DLL se hatao
        map.remove(key);        // map se hatao
        return -1;              // expired = not
found
    }
    moveToFront(node);
    return node.value;
}

public void put(int key, int value, long ttlMillis)
{
    // ... classic logic same ...
    fresh.expiryTime = System.currentTimeMillis() +
ttlMillis;
}
```

Lazy expiry ka problem (interviewer poochhega hi): Agar koi expired key kabhi access hi na ho, toh wo memory me padi rahegi aur **capacity block karti rahegi** — ek valid entry ki jagah kha jaayegi. Ye seedha bolna, phir solution do.

Do better solutions:

1. Background cleanup thread — ek ScheduledExecutorService har X second pe chal ke expired entries saaf kare. Simple, but poore map pe scan karna pad sakta hai.

2. Min-heap by expiryTime — ek `PriorityQueue<Node>` jisme sabse jaldi expire hone wala top pe ho. Cleanup ke time sirf top dekho — agar wo expired nahi, toh koi bhi expired nahi. Ye O(log n) insert, but cleanup bahut efficient.

3. Eviction ke waqt bhi check — jab capacity full ho aur evict karna ho, pehle tail se koi expired node dhundh lo aur usko nikaalo (valid LRU node bachane ke liye).

Interview me ye framing use karo: "Lazy expiry se shuru karunga kyunki simple aur

O(1) hai, phir agar memory ek concern hai toh min-heap ya background sweeper add karunga. Trade-off hai simplicity vs memory-efficiency."

🗣 Khud Test Kar

Q: Min-heap approach me cleanup fast kyun hai?

> A: Heap ka top hamesha sabse jaldi expire hone wala hota hai. Agar top expired nahi hai, toh baaki koi bhi expired nahi ho sakta — poora scan karne ki zaroorat hi nahi.

12 LRU Variation B — Heaps + Maps (DLL ke Bina)

Doosre candidate ko LRU ***heaps + maps*** *se implement karne ko bola gaya — classic DLL approach allowed nahi thi. Ye deliberately tricky hai, kyunki interviewer dekhna chahta hai ki tu trade-offs samajhta hai ya sirf ratta maara hua pattern likh raha hai.*

Idea: Har entry ke saath ek **timestamp/counter** rakho (kab last access hua). Min-heap us timestamp pe ordered ho — top pe sabse purana. Evict karna ho toh heap ka top nikaal do.

- `HashMap<Integer, Entry>` → O(1) lookup
- `PriorityQueue<Entry>` ordered by lastAccessTime → LRU find karne ke liye

```
class Entry {
    int key, value;
    long accessCounter;   // monotonic counter,
timestamp se safer
}

class LRUCacheHeap {
    private final Map<Integer, Entry> map = new
HashMap<>();
    private final PriorityQueue<Entry> heap =
        new PriorityQueue<>((a, b) ->
            Long.compare(a.accessCounter,
b.accessCounter));
```

```
    private long counter = 0;
    private final int capacity;

    public int get(int key) {
        Entry e = map.get(key);
        if (e == null) return -1;
        // "touch" — heap se purani entry hatao,
nayi daalo
        heap.remove(e);            // O(n) — yahi
bottleneck hai
        e.accessCounter = ++counter;
        heap.offer(e);             // O(log n)
        return e.value;
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            Entry e = map.get(key);
            e.value = value;
            heap.remove(e);
            e.accessCounter = ++counter;
            heap.offer(e);
            return;
        }
        if (map.size() == capacity) {
            Entry lru = heap.poll();      // O(log
n)
            map.remove(lru.key);
        }
        Entry fresh = new Entry(key, value,
++counter);
        map.put(key, fresh);
        heap.offer(fresh);
    }
}
```

Ye version DLL se WORSE hai — aur yehi interviewer sunna chahta hai! `heap.remove(element)` O(n) hai (element dhundhna padta hai), isliye get() O(n) ho gaya — jabki DLL version me O(1) tha. Agar tu ye trade-off khud se bolega, tu jeet gaya.

Optimization — "lazy deletion" trick: Heap se remove mat karo. Bas nayi entry daal do purane counter ke saath, aur purani ko "stale" maano. Jab poll karo, check karo ki heap ka top map me current entry se match karta hai ya nahi — agar nahi, wo stale hai, discard karke agla poll karo.

Isse get() O(log n) ho jaata hai (O(n) ke bajaye), memory thodi zyada lagti hai.

Interview me ye bolna: "Heap-based LRU possible hai, lekin get() O(log n) ya O(n) ho jaata hai jabki DLL+HashMap se O(1) milta hai. Main DLL prefer karunga, unless requirement me kuch aisa ho jo heap zaroori banaye — jaise TTL-based ordering bhi chahiye."

🧠 Khud Test Kar

Q: Heap-based LRU me sabse bada bottleneck kya hai?

> A: heap.remove(element) O(n) hai kyunki heap me arbitrary element dhundhna padta hai — heap sirf top pe efficient hai. Isse get() O(1) se O(n) ho jaata hai.

13 Session Manager (HashMap + TTL)

*Login karne pe server ek **session ID** deta hai. Har request pe ye ID aati hai, aur server ko turant batana hai — "ye kaun hai, valid hai ya expire ho gaya." Ye lookup lakhon baar hoti hai, isliye O(1) chahiye.*

```
class Session {
    String userId;
    long createdAt, expiresAt;
}

class SessionManager {
    private final Map<String, Session> sessions
        = new ConcurrentHashMap<>();

    public String createSession(String userId, long
ttlMillis) {
        String sessionId =
UUID.randomUUID().toString();
        long now = System.currentTimeMillis();
        sessions.put(sessionId,
            new Session(userId, now, now +
ttlMillis));
        return sessionId;
    }

    public boolean isValid(String sessionId) {
        Session s = sessions.get(sessionId);
        if (s == null) return false;
        if (System.currentTimeMillis() >
```

```
s.expiresAt) {
            sessions.remove(sessionId);   // lazy
expiry
            return false;
        }
        return true;
    }

    public void invalidate(String sessionId) {
        sessions.remove(sessionId);
    }
}
```

Do design decisions jo interviewer definitely poochhega:
• **Thread safety** → `ConcurrentHashMap`, plain HashMap nahi (multiple requests parallel aayengi)
• **Expiry strategy** → lazy (simple) vs background sweeper vs min-heap — bilkul wahi trade-off jo LRU+TTL me tha

Scale-up follow-ups:
• **"Multiple servers pe kaise?"** → in-memory map kaam nahi karega (server A ka session server B ko nahi pata). Solution: Redis jaisa shared store, ya stateless JWT tokens.
• **"Sliding expiry chahiye?"** → har access pe `expiresAt` refresh kar do — matlab active user ka session expire na ho.

🧠 Khud Test Kar

Q: Multi-server setup me in-memory session map kyun fail karta hai?

> A: Load balancer request kisi bhi server pe bhej sakta hai. Server A pe bana session server B ki memory me nahi hota, isliye user random logout dikhega. Shared store (Redis) ya stateless token chahiye.

14 Time Window Map / Rate Counter

Track karna hai — "pichhle 5 minute me kitne events aaye?" Purane events apne-aap window se bahar ho jaane chahiye. Ye **rate limiting** *aur monitoring dashboards ka core*

pattern hai.

```
class TimeWindowCounter {
    private final Deque<Long> timestamps = new
ArrayDeque<>();
    private final long windowMillis;

    public TimeWindowCounter(long windowMillis) {
        this.windowMillis = windowMillis;
    }

    public synchronized void recordEvent() {
        long now = System.currentTimeMillis();
        timestamps.addLast(now);
        cleanup(now);
    }

    public synchronized int countInWindow() {
        cleanup(System.currentTimeMillis());
        return timestamps.size();
    }

    private void cleanup(long now) {
        while (!timestamps.isEmpty()
                && now - timestamps.peekFirst() >
windowMillis) {
            timestamps.pollFirst();
        }
    }
}
```

Front se hi cleanup kyun: Hum hamesha time-order me insert karte hain (naya tail pe), isliye front hamesha sabse purana hota hai. Front se removal O(1), beech me dhundhna O(n) hota.

Memory trap: Agar har event ka timestamp store karoge aur traffic bahut high ho (millions/sec), Deque phat jaayega. **Better production answer — bucketing:** window ko chhote buckets me todo (jaise 5 min = 300 buckets of 1 sec), har bucket me sirf ek **count** rakho. Memory O(number of buckets) ho jaayegi, events ki sankhya se independent.

🧠 Khud Test Kar

Q: Bucketing approach memory kaise bachata hai?

> A: Har individual event ka timestamp store

karne ke bajaye, per-second (ya per-bucket) sirf ek counter rakhte hain. 1 million events bhi ek bucket me sirf ek integer hi lenge.

15 Number of Atoms (LeetCode 726, Hard)

Chemical formula string di hai jaise `"K4(ON(SO3)2)2"`, har element ka total count nikaalna hai, output alphabetically sorted. Nested brackets ke andar ka sab kuch bahar wale multiplier se multiply hota hai.

Intuition: Har `(` pe naya "scope" (map) kholo, har `)` pe use band karke uske saare counts ko multiplier se multiply karke bahar wale scope me merge kar do — nested folders jaisa.

```
public String countOfAtoms(String formula) {
    int n = formula.length(), i = 0;
    Deque<Map<String, Integer>> stack = new
ArrayDeque<>();
    stack.push(new HashMap<>());

    while (i < n) {
        char ch = formula.charAt(i);
        if (ch == '(') {
            stack.push(new HashMap<>());
            i++;
        } else if (ch == ')') {
            Map<String, Integer> closed =
stack.pop();
            i++;
            int start = i;
            while (i < n &&
Character.isDigit(formula.charAt(i))) i++;
            int mult = (start == i) ? 1
                :
Integer.parseInt(formula.substring(start, i));
            Map<String, Integer> parent =
stack.peek();
            for (Map.Entry<String, Integer> e :
closed.entrySet())
                parent.merge(e.getKey(),
                             e.getValue() * mult,
Integer::sum);
        } else {
            int start = i;
            i++;   // uppercase
            while (i < n &&
Character.isLowerCase(formula.charAt(i))) i++;
```

```
            String element =
formula.substring(start, i);
            int numStart = i;
            while (i < n &&
Character.isDigit(formula.charAt(i))) i++;
            int count = (numStart == i) ? 1
                :
Integer.parseInt(formula.substring(numStart, i));
            stack.peek().merge(element, count,
Integer::sum);
        }
    }

    TreeMap<String, Integer> sorted = new TreeMap<>
(stack.pop());
    StringBuilder sb = new StringBuilder();
    for (Map.Entry<String, Integer> e :
sorted.entrySet()) {
        sb.append(e.getKey());
        if (e.getValue() > 1)
sb.append(e.getValue());
    }
    return sb.toString();
}
```

Step	Action	Stack (top→bottom)
1	K4	[{K:4}]
2	(	[{}, {K:4}]
3	O, N	[{O:1,N:1}, {K:4}]
4	(	[{}, {O:1,N:1}, {K:4}]
5	S, O3	[{S:1,O:3}, {O:1,N:1}, {K:4}]
6	)2 → merge	[{O:7,N:1,S:2}, {K:4}]
7	)2 → merge	[{K:4,O:14,N:2,S:4}]

Trap: Multiplier **poore closed map** pe lagta hai — jisme nested se already-merged counts bhi shamil hain. Step 6 me O already 7 tha (1 direct + 6 nested), aur poora 7 hi ×2 hua. Sirf direct elements pe multiply karna sabse common bug hai.

Complexity: Time O(n) single pass, Space O(n) stack + maps.

16 tail -f Implementation

Linux ka `tail -f logfile` file ke end pe "chipka" rehta hai — naye lines likhte hi turant

dikhate hain, bina poori file dobara padhe.

Core idea: Ek **offset (file pointer)** yaad rakho. Periodically dekho file ka size badha kya — agar haan, sirf **naya hissa** (offset se end tak) padho, print karo, offset update karo.

```
public void tail(String filePath) throws Exception
{
    RandomAccessFile file = new
RandomAccessFile(filePath, "r");
    long filePointer = file.length();   // end se
shuru

    while (true) {
        long currentLength = file.length();
        if (currentLength < filePointer) {
            // file truncate/rotate ho gayi
            filePointer = 0;
        }
        if (currentLength > filePointer) {
            file.seek(filePointer);
            String line;
            while ((line = file.readLine()) !=
null)
                System.out.println(line);
            filePointer = file.getFilePointer();
        }
        Thread.sleep(500);
    }
}
```

Do follow-ups jo pakka aayenge:

• **"Polling ke bajaye better?"** → OS-level file events — Linux me `inotify`, Java me `WatchService` API. Event-driven, CPU waste nahi hota.

• **"Log rotation handle karoge?"** → Agar file size achanak **ghat** jaaye, matlab file rotate/truncate hui — pointer 0 pe reset karo (upar code me handle hai). Advanced: file ka inode track karo.

🧠 Khud Test Kar

Q: Poori file baar-baar kyun nahi padhte?

> A: File GBs ki ho sakti hai. Offset yaad rakh ke sirf naya delta padhna O(naya data) hai, O(poori file) nahi.

17 Large File — Bina Pura Load Kiye

50 GB ki file, 8 GB RAM. `Files.readAllLines()` *use kiya toh* **OutOfMemoryError**. *Solution: streaming — ek time pe thoda hissa.*

Case A — Sequential processing (line by line): `BufferedReader` internally chhota buffer (~8KB) rakhta hai, poori file nahi. Memory constant rehti hai.

```
try (BufferedReader reader =
         new BufferedReader(new FileReader(path)))
{
    String line;
    while ((line = reader.readLine()) != null) {
        process(line);   // ek line, phir free
    }
}
```

Case B — Last N lines (harder): File **end se** padhni hai, but files forward-read ke liye bani hoti hain. Trick: `RandomAccessFile` se end se peeche ki taraf chunk-by-chunk padho, newlines count karo, jab tak N na mil jaayein.

```
public List<String> lastNLines(String path, int n)
        throws IOException {
    List<String> result = new ArrayList<>();
    try (RandomAccessFile file =
             new RandomAccessFile(path, "r")) {
        long pointer = file.length() - 1;
        StringBuilder sb = new StringBuilder();

        while (pointer >= 0 && result.size() < n) {
            file.seek(pointer);
            char c = (char) file.read();
            if (c == '\n') {
                if (sb.length() > 0) {

result.add(sb.reverse().toString());
                    sb.setLength(0);
                }
            } else {
                sb.append(c);
            }
            pointer--;
        }
        if (sb.length() > 0 && result.size() < n)
            result.add(sb.reverse().toString());
    }
```

```
    Collections.reverse(result);   // original
order
    return result;
}
```

Ye zaroor bolna: "Space complexity O(1) hai — file size se **independent**." Sirf "time O(n)" bolna kaafi nahi; memory-independence hi wo cheez hai jo interviewer test kar raha hai.

Optimization: Byte-by-byte padhna slow hai (har read pe syscall). Production me 4KB/8KB **chunks** me padho aur us buffer ke andar newlines count karo.

🧠 Khud Test Kar

Q: Last N lines ke liye start se kyun nahi padhte?

> A: Start se padhein toh poori file traverse karni padegi ye jaanne ke liye ki last N kahan se shuru hui — O(file size). End se peeche padhne me sirf utna kaam hota hai jitna N lines ke liye chahiye.

18 Map + Tree DSA — Pattern Guide

Ek candidate ne bataya Round 2 me "Map + Tree based" question tha, jisme **complexity aur test cases pe explicit focus** *tha. Ye ek specific question nahi, ek* **pattern family** *hai — inhe pehchanna important hai.*

Common patterns is family me:

- **Vertical order traversal** → `TreeMap<column, List<node>>`
- **Clone a tree/graph** → `Map<oldNode, newNode>` visited-tracking ke liye
- **Build tree from parent-child pairs** → `Map<id, Node>` se nodes lookup
- **Path sum with prefix counts** → `Map<prefixSum, count>`
- **Lowest Common Ancestor with parent map** → `Map<child, parent>`
- **Subtree deduplication** →

`Map<serializedSubtree, count>`

```
// Example: Vertical Order Traversal
public List<List<Integer>> verticalOrder(TreeNode
root) {
    if (root == null) return new ArrayList<>();
    TreeMap<Integer, List<Integer>> map = new
TreeMap<>();
    Queue<TreeNode> nodes = new LinkedList<>();
    Queue<Integer> cols = new LinkedList<>();
    nodes.offer(root); cols.offer(0);

    while (!nodes.isEmpty()) {
        TreeNode node = nodes.poll();
        int col = cols.poll();
        map.computeIfAbsent(col, k -> new
ArrayList<>())
           .add(node.val);
        if (node.left != null) {
            nodes.offer(node.left); cols.offer(col
- 1);
        }
        if (node.right != null) {
            nodes.offer(node.right); cols.offer(col
+ 1);
        }
    }
    return new ArrayList<>(map.values());
}
```

Kyunki is round me complexity + test cases pe zor tha, har solution ke baad ye 4 cheezein bolne ki aadat daalo:

1. Time complexity + kyun
2. Space complexity + kyun
3. **Edge cases**: null root, single node, skewed tree (sab left ya sab right), duplicate values
4. Ek alternative approach aur uska trade-off

HashMap vs TreeMap ka decision:
HashMap O(1) lookup deta hai but **order nahi**. TreeMap O(log n) hai but keys **sorted** rehti hain. Agar output me sorted order chahiye (jaise vertical traversal me columns left-to-right), TreeMap use karo — warna HashMap + baad me sort bhi chalega.

🧠 Khud Test Kar

Q: Vertical order traversal me TreeMap kyun, HashMap kyun nahi?

A: Columns ko left-se-right sorted order me output karna hai. TreeMap keys automatically sorted rakhta hai; HashMap ke saath end me alag se sort karna padta.

PART 4

System Design (HLD)

Round open-ended hai — framework yaad rakhna, ratta nahi

19 Requirement Gathering Framework

Sabse important insight: Dono candidates ne bataya HLD round **open-ended** tha. Matlab interviewer ek line me problem dega ("design a key-value store") aur **chup ho jaayega**. Tera evaluation isi baat pe hoga ki tu **sahi sawaal poochta hai ya nahi** — turant solution likhna sabse badi galti hai.

Pehle 5-7 minute sirf sawaal poochne me lagao. Ye 4 categories:

1. Functional — Kya operations chahiye? (get/put/delete? range queries? transactions?)
2. Scale — Kitne users? QPS kya? Data size? Read-heavy ya write-heavy?
3. Non-functional — Latency target? Availability (99.9% ya 99.99%)? Consistency kitni strict?
4. Constraints — Single datacenter ya multi-region? Cost matter karta hai? Existing tech stack?

Ye exact opening line use kar sakta hai:
"Solution pe jaane se pehle main kuch requirements clarify karna chahunga, taaki main sahi trade-offs choose kar sakun. Pehla — is system ka primary use-case kya hai, aur expected scale kya hai?"

Uske baad ka flow (ye order follow karo):
1. Requirements + scale estimate (numbers likho — QPS, storage)
2. High-level architecture (boxes and arrows)
3. Data model / API design
4. Deep dive ek-do components me (interviewer choose karega)
5. Bottlenecks + trade-offs discuss karo
6. Scale/failure scenarios (ek node mar jaaye toh?)

20 Design a Key-Value Store

Ye exact question ek candidate ko mila tha. Distributed KV store = DynamoDB/Cassandra/Redis jaisa system. Ye Confluent ke liye extra relevant hai kyunki Kafka khud partitioning aur replication pe chalta hai — same concepts.

Step 1 — Requirements clarify karo:
• Operations: get(key), put(key, value), delete(key)? Range scan chahiye?
• Value size: few KB ya MBs?
• Scale: kitna data (TB/PB)? QPS?
• Consistency: strong ya eventual chalega?
• Durability: data loss bilkul acceptable nahi?

Step 2 — Single node se shuru karo, phir scale karo. Interviewer ko dikhao ki tu incremental soch raha hai:

"Single machine pe ye ek HashMap + write-ahead log hai. Problem: (a) memory limit, (b) machine mar gayi toh sab gaya. Isliye distribute karna padega."

Step 3 — Partitioning (Sharding):
Data ko multiple nodes me baantna. Simple approach: `hash(key) % N`.

Problem: Agar N badle (node add/remove), **saari keys remap ho jaati hain** — poora data move karna padega.

Solution — Consistent Hashing: Nodes aur keys dono ko ek "hash ring" pe rakho. Key us node pe jaati hai jo ring me clockwise sabse pehle mile. Node add/remove hone pe sirf **uske padosi ka data** move hota hai, sab ka nahi.

Virtual nodes: Har physical node ko ring pe kai jagah rakho — isse load evenly distribute hota hai.

Step 4 — Replication:
Har key ko N nodes pe rakho (typically N=3), ring me agle N nodes pe. Ek node mare toh data safe hai.

Quorum (W + R > N):
• W = kitne nodes pe likhna confirm ho tab success bolein
• R = kitne nodes se padhna tab result dein
• Agar W + R > N, toh guaranteed hai ki read me kam se kam ek latest copy milegi
• Example N=3, W=2, R=2 → balanced. W=1 → fast writes, weak consistency. W=3 → slow but strong.

Step 5 — Consistency (CAP theorem):
Network partition hoga hi (P is mandatory). Toh choose karna hai:
• **CP** (Consistency) — partition me kuch requests reject karo, but data hamesha correct (jaise banking)
• **AP** (Availability) — hamesha respond karo, data thoda purana ho sakta hai (jaise social feed)

Conflict resolution: Agar do nodes pe alag values likhi gayin — **Vector clocks** (kaunsi version kis se aayi track karo) ya **Last-Write-Wins** (timestamp se, simple but data loss possible).

Step 6 — Storage engine:
• **LSM Tree** (Cassandra, RocksDB) — writes memory buffer me, phir sequentially disk pe flush. **Write-heavy ke liye fast.**
• **B-Tree** (traditional DBs) — in-place updates. **Read-heavy ke liye better.**

Kafka bhi append-only log use karta hai — sequential writes hamesha random writes se tez hote hain.

Step 7 — Failure handling:

• **Gossip protocol** — nodes ek doosre ko periodically ping karke membership/health share karte hain
• **Hinted handoff** — target node down hai toh koi aur node temporarily write rakh le, baad me deliver kare
• **Merkle trees** — do replicas ka data compare karke sirf difference sync karo (anti-entropy repair)

Confluent-specific connection banana (bonus points):
"Ye same concepts Kafka me bhi hain — partitions = sharding, replicas + ISR = replication, acks=all ≈ quorum write, aur log-structured append-only storage. KORA me ye multi-tenant scale pe extend hota hai."

🗣️ Khud Test Kar

Q: Consistent hashing simple modulo se better kyun hai?

> A: Modulo me node count badalne pe lagbhag saari keys remap ho jaati hain (massive data movement). Consistent hashing me sirf ring ke padosi ka hissa move hota hai — roughly 1/N data.

Q: W + R > N kya guarantee deta hai?

> A: Read aur write ke node-sets me kam se kam ek node common hoga — matlab read me guaranteed latest write dikhega.

PART 5

Kafka + Domain Knowledge

Role-specific — yahi tujhe baaki candidates se alag karega

21 Kafka Core Internals — Quick Revision

Concept	Ek line me
Topic / Partition	Topic logical naam; partition actual ordered log. Order sirf partition ke andar guaranteed hai.
Replication / ISR	Har partition ki N copies. ISR = In-Sync Replicas — jo leader ke saath up-to-date hain.
acks	acks=0 (fire-forget), 1 (leader ne likha), all (saare ISR ne likha — safest)
Consumer Group	Ek group ke andar har partition sirf ek consumer ko. Parallelism = partition count.
Rebalancing	Consumer add/remove hone pe partitions dobara baante jaate hain. Stop-the-world hota hai (naya protocol isko improve karta hai).
Offset	Consumer kahan tak padh chuka. Commit karne pe hi progress save hoti hai.
Log Compaction	Har key ki sirf latest value rakho — changelog/state store ke liye.
Exactly-once	Idempotent producer + transactions se duplicate na ho.

Sabse common interview question: "Kafka ordering kaise guarantee karta hai?" — Answer: **Sirf partition ke andar**. Agar poore topic me ordering chahiye toh single partition (throughput ki keemat pe), ya same key ke messages same partition me bhejo (key-based partitioning).

22 KORA — Confluent ka Cloud-Native Engine

*Self-managed Kafka chalana ek full-time job hai — brokers, disk, scaling, recovery sab khud dekho. Confluent ne socha "customer ko Kafka **use** karna chahiye, **chalana** nahi" — isi se bana **KORA**, jo Apache Kafka ka lagbhag poora rewrite hai (~20% code hi original se match karta hai).*

Core concepts:

• **Logical Kafka Cluster** — customer jo dekhta/manage karta hai; peeche shared physical infra

• **CKU (Confluent Kafka Unit)** — capacity + billing ka unit

• **Control Plane vs Data Plane** — config/decisions alag, actual data movement alag

• **Multi-tenancy** — hazaaron customers ek shared cluster pe, safely isolated

• **Tiered Storage** — purana data cheap object storage pe, "infinite retention" bina cost explode kiye

• **Self-Balancing Clusters** — load khud redistribute hota hai, manual intervention nahi

23 Marketplace & Hub — Tera Actual Domain

Kafka ko Salesforce/MongoDB/Snowflake se connect karna ho toh har baar khud connector likhna painful hoga. Isliye Confluent ek **"App Store"** *jaisa system chalata hai — pre-built connectors browse karo, install karo, use karo.*

JD ke exact 4 responsibilities:
1. Catalog — kaunse connectors/artifacts available hain, metadata, versions
2. Publishing — naya connector system me register karna
3. Distribution — customer ke Kafka/Flink environment tak deliver karna
4. Lifecycle Management — versions, deprecation, backward-compatibility

Scope clear rakhna: Tu poore KORA broker-internals (replication protocol wagera) pe kaam nahi karega — tera scope **Marketplace & Hub ka ek well-defined portion** hai, jo KORA ecosystem ke andar ek component ki tarah deploy hota hai.

Agar poochhein "is role me kya interest hai" toh ye angle use kar: "Mera background distributed systems aur Kafka pe hai — aur ye role usko ek platform/product perspective deta hai, jahan hazaaron customers ke liye artifacts ka lifecycle manage karna hota hai. Ye scale ka problem mujhe interesting lagta hai."

PART 6

Behavioral Round + STAR

Recorded video assessment ke liye bhi yahi kaam aayega

24 STAR Framework aur Ready Stories

STAR:
Situation — context kya tha (15 sec)
Task — teri responsibility kya thi (15 sec)
Action — tune kya kiya, specifically (60-90 sec, sabse bada hissa)
Result — outcome, numbers ke saath agar possible (20 sec)

Sabse common galti: "Hum ne ye kiya" bolna. Interviewer **tera** contribution sunna chahta hai — "**Maine** ye design kiya, **maine** ye decision liya." Team ka credit dena achha hai, but apna role clear rakhna.

JD ke values se match karti hui 5 stories tayaar rakh:

1. "Root cause tak gaya, symptom fix nahi kiya" — JD me literally likha hai: "*Identify root causes, and get beyond treating symptoms*". Koi production bug jise tune deep debug karke asli wajah dhundhi.

2. "Complex project delivered with quality" — EVA-AI ya koi bada feature, jisme tests/logging/error-handling ka dhyan rakha.

3. "Customer-focused decision" — jab tune customer impact ke liye extra mile gaya ya scope adjust kiya.

4. "Team ke liye decision, apne liye nahi" — jab tune apna preferred approach chhoda kyunki team/company ke liye behtar tha.

5. "Disagreement handle kiya" — technical

disagreement jahan tune data se argue kiya, ego se nahi.

Recorded video assessment ke liye extra tips:
• Ek hi take milta hai — pehle likh ke practice kar, phir bolke
• ~30 sec sochne ka time, phir ~3 min record
• Camera me dekh, screen me nahi
• Quiet + well-lit jagah, Chrome browser
• Phone se khud ko record karke playback dekh — filler words pakad me aayenge

Naye-zamane ka question jo aa raha hai:
"AI tools ka use kaise karte ho, aur output verify kaise karte ho?" — Tere paas EVA-AI ka production AI-integration experience hai, ye story yahan perfect fit hai. Bolna ki tu AI output ko review/test karta hai, blindly trust nahi.

FINAL

Cheat Sheet — Din 4 Ki Raat

Interview se pehle sirf ye padhna kaafi hai

⚡ Concurrency

- **Pehla step hamesha:** shared mutable state identify karo
- synchronized (simple, auto-release) vs ReentrantLock (tryLock, timeout, fairness)
- unlock() hamesha finally me
- volatile = visibility only; Atomic = visibility + atomicity (CAS)
- ExecutorService: Runnable (void) vs Callable (returns Future); shutdown() bhoolna mat
- ConcurrentHashMap > synchronizedMap (fine-grained locking); putIfAbsent/computeIfAbsent atomic hain
- Deadlock fix: lock ordering, tryLock+timeout, chhota lock scope

🧩 Coding Problems

- **LRU** = HashMap + DLL, dummy sentinels, Node me key bhi rakho. get/put O(1)
- **LRU+TTL** = lazy expiry se shuru, phir min-heap/background sweeper ka trade-off bolo
- **LRU heap version** = worse hai (get O(n) due to heap.remove) — ye khud bolna
- **Ring buffer** = head/tail + modulo, resize se amortized O(1), random() = direct index
- **Sudoku MT** = 27 independent read-only tasks, no lock needed, AtomicBoolean for early exit
- **Job Scheduler** = PriorityBlockingQueue + dispatcher + worker pool; DelayQueue better than polling
- **Time Window** = Deque of timestamps; high traffic pe bucketing use karo
- **tail -f** = offset track, sirf delta padho; inotify/WatchService > polling; rotation handle

karo

- **Large file** = BufferedReader streaming; last-N = RandomAccessFile backward; space O(1) bolna
- **Number of Atoms** = stack of maps, multiplier poore closed map pe lagta hai

🏗 System Design

- **Turant solution mat likho** — 5-7 min requirements poochho (functional, scale, non-functional, constraints)
- Single node → phir scale, incremental sochna dikhao
- Consistent hashing + virtual nodes (modulo se better kyun — bolna)
- Quorum: W + R > N guarantees latest read
- CAP: partition to hoga hi — CP ya AP choose karo, kyun bhi batao
- LSM tree (write-heavy) vs B-tree (read-heavy)
- Failure: gossip, hinted handoff, Merkle trees
- Kafka se connection banao — partitions=sharding, ISR=replication, acks=all≈quorum

🎤 Behavioral

- STAR — Action sabse lamba hissa (60-90 sec)
- "Maine" bolo, sirf "humne" nahi
- Root-cause story zaroor tayaar rakho (JD me explicitly hai)
- AI-tools verification wali story — EVA-AI experience use karo

✅ Interview Din Ka Checklist

- Har solution ke baad: time + space complexity khud se bolo, poochne ka wait mat karo
- Edge cases khud batao: null, empty, single

element, duplicates, overflow

- Thread-safety follow-up har coding question me aa sakta hai — ready raho
- Code likhne se pehle approach 30 sec me bolo, phir likho
- Atak jao toh chup mat raho — loudly soch, interviewer hint dega